

Федеральное государственное автономное образовательное учреждение высшего
Образования «Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Лабораторная работа №5 по программированию
Вариант 3300

Работу выполнил:

Прохоревич Иван Борисович

Группа Р3110

Преподаватель, принимающий работу:

Выдра Андрей Михайлович

Преподаватель факультета ПИиКТ

г. Санкт-Петербург 2026 г.

Задание

Реализовать консольное приложение, которое реализует управление коллекцией объектов в интерактивном режиме. В коллекции необходимо хранить объекты класса `Vehicle`, описание которого приведено ниже.

Разработанная программа должна удовлетворять следующим требованиям:

- Класс, коллекцией экземпляров которого управляет программа, должен реализовывать сортировку по умолчанию.
- Все требования к полям класса (указанные в виде комментариев) должны быть выполнены.
- Для хранения необходимо использовать коллекцию типа `java.util.PriorityQueue`
- При запуске приложения коллекция должна автоматически заполняться значениями из файла.
- Имя файла должно передаваться программе с помощью: аргумент командной строки.
- Данные должны храниться в файле в формате `csv`
- Чтение данных из файла необходимо реализовать с помощью класса `java.io.BufferedReader`
- Запись данных в файл необходимо реализовать с помощью класса `java.io.OutputStreamWriter`
- Все классы в программе должны быть задокументированы в формате `javadoc`.
- Программа должна корректно работать с неправильными данными (ошибки пользовательского ввода, отсутствие прав доступа к файлу и т.п.).

В интерактивном режиме программа должна поддерживать выполнение следующих команд:

- `help` : вывести справку по доступным командам
- `info` : вывести в стандартный поток вывода информацию о коллекции (тип, дата инициализации, количество элементов и т.д.)
- `show` : вывести в стандартный поток вывода все элементы коллекции в строковом представлении
- `add {element}` : добавить новый элемент в коллекцию
- `update id {element}` : обновить значение элемента коллекции, `id` которого равен заданному
- `remove_by_id id` : удалить элемент из коллекции по его `id`
- `clear` : очистить коллекцию
- `save` : сохранить коллекцию в файл
- `execute_script file_name` : считать и исполнить скрипт из указанного файла. В скрипте содержатся команды в таком же виде, в котором их вводит пользователь в интерактивном режиме.
- `exit` : завершить программу (без сохранения в файл)
- `remove_head` : вывести первый элемент коллекции и удалить его
- `remove_lower {element}` : удалить из коллекции все элементы, меньшие, чем заданный
- `history` : вывести последние 13 команд (без их аргументов)
- `count_greater_than_fuel_type fuelType` : вывести количество элементов, значение поля `fuelType` которых больше заданного
- `print_ascending` : вывести элементы коллекции в порядке возрастания
- `print_unique_fuel_type` : вывести уникальные значения поля `fuelType` всех элементов в коллекции

Формат ввода команд:

- Все аргументы команды, являющиеся стандартными типами данных (примитивные типы, классы-оболочки, String, классы для хранения дат), должны вводиться в той же строке, что и имя команды.
- Все составные типы данных (объекты классов, хранящиеся в коллекции) должны вводиться по одному полю в строку.
- При вводе составных типов данных пользователю должно показываться приглашение к вводу, содержащее имя поля (например, "Введите дату рождения:")
- Если поле является enum'ом, то вводится имя одной из его констант (при этом список констант должен быть предварительно выведен).
- При некорректном пользовательском вводе (введена строка, не являющаяся именем константы в enum'e; введена строка вместо числа; введённое число не входит в указанные границы и т.п.) должно быть показано сообщение об ошибке и предложено повторить ввод поля.
- Для ввода значений null использовать пустую строку.
- Поля с комментарием "Значение этого поля должно генерироваться автоматически" не должны вводиться пользователем вручную при добавлении.

Описание хранимых в коллекции классов:

```
public class Vehicle {
    private int id; //Значение поля должно быть больше 0, Значение этого поля должно быть
    уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.util.Date creationDate; //Поле не может быть null, Значение этого поля должно
    генерироваться автоматически
    private Float enginePower; //Поле не может быть null, Значение поля должно быть больше 0
    private long numberOfWheels; //Значение поля должно быть больше 0
    private VehicleType type; //Поле не может быть null
    private FuelType fuelType; //Поле не может быть null
}

public class Coordinates {
    private Double x; //Значение поля должно быть больше -371, Поле не может быть null
    private Float y; //Максимальное значение поля: 376, Поле не может быть null
}

public enum VehicleType {
    HELICOPTER,
    MOTORCYCLE,
    CHOPPER,
    HOVERBOARD;
}

public enum FuelType {
    KEROSENE,
    MANPOWER,
    NUCLEAR,
    PLASMA,
    ANTIMATTER;
}
```


Исходный код программы

https://github.com/ivanrevich/ITMO_Labs/tree/main/programming/lab2_5

Результат работы программы

```
>> add
Enter vehicle name:
>> Helicopter
Enter vehicle engine power:
>> 12
Enter vehicle number of wheels
>> 2
Choose vehicle type:
1: HELICOPTER
2: MOTORCYCLE
3: CHOPPER
4: HOVERBOARD

>> 1
Choose fuel type:
1: KEROSENE
2: MANPOWER
3: NUCLEAR
4: PLASMA
5: ANTIMATTER

>> 1
Enter x:
>> 122
Enter y:
>> 9
>> show
Vehicle{id=0, name='Helicopter', coordinates=Coordinates{x=122.0, y=9.0}, creationDate=Sat Mar 14 12:45:02 MSK 2026, enginePower=12.0, numberOfWheels=2, type=HELICOPTER, fuelType=KEROSENE}
>> history
add
show
>> save
>> exit
```

Выводы

В результате выполнения данной работы я написал продвинутое, хорошо расширяемое консольное приложение с использованием нескольких паттернов проектирования. Также поработал с коллекциями, steam и лямбда выражениями, и еще использовал generic-и впервые.